10-docker-realtime.md

Docker Compose — ELI5 + Real World Map 🗺️

🏘️ Big Picture First

Tool	Real World	What it does
Docker Run	One cook, one dish	Single container
Docker Compose	One kitchen, many dishes	Multiple containers, one server
Docker Swarm	Many kitchens, many dishes	Multiple containers, many servers

> **Simple rule:** Compose = One hotel kitchen running multiple counters (pizza, biryani, burger) at the same time

🗺️ The Flow Map

```
You write ONE yaml file
        ↓
docker compose up
        ↓
  ┌──────────────────────────────┐
  │  Container 1 → App (nginx)   │
  │  Container 2 → DB (mysql)    │
  │  Container 3 → Cache         │
  └──────────────────────────────┘
All running on ONE server
```

📄 The YAML File — ELI5

Think of `docker-compose.yaml` like a **hotel menu card** — it lists every counter, what they serve, and which door (port) to use.

yaml

```yaml
version: "3"          # which compose version — just keep "3"


services:             # list of your counters/stalls
  pubg:               # counter name (you decide)
    image: nginx      # what to cook (which Docker image)
```

```
    ports:
      - "8081:80"     # your door : container's door
                                          # localhost:8081 → nginx:80


  swiggy:
    image: httpd
    ports:
      - "8082:80"
```

Port mapping memory trick:

```
"HOST:CONTAINER"
"YOUR DOOR : THEIR DOOR"
8081:80  →  you visit 8081, container listens on 80
```

🏗️ image vs build — The Key Difference

	`image:`	`build:`
What	Use a ready-made image	Build from your Dockerfile
When	Code not changing	Your own custom app
Real world	Order from Zomato	Cook at home

yaml

```
# Using ready image (order from Zomato)

services:
  app:
    image: nginx

# Building your own (cook at home)

services:
  app:
    build: ./movie/    # path to your Dockerfile folder
```

🎬 The Movie & Train Example — Story Style

```
You have 2 apps:
  🎥  movie/ → has Dockerfile + index.html
  🚂  train/ → has Dockerfile + index.html
```

yaml

```yaml
services:
  movie:
    build: ./movie/    # go inside movie folder, build it
    ports:
            - "8084:80"


  train:
    build: ./train/    # go inside train folder, build it
    ports:
      - "8085:80"
```

bash

```bash
docker compose up --build -d
# --build = rebuild images if code changed

# -d      = run in background (detached)
```

> **Key lesson:** Every time you change code → must rebuild! `docker compose up --build -d` does BOTH in one shot.

🔴 The Big Gotcha — Snapshot Problem

```
You change movie/index.html
         ↓
curl localhost:8084  ← still shows OLD content 😱
         ↓
Why? Docker image is a SNAPSHOT, not live code
         ↓
Fix: docker compose up --build -d  ← rebuilds fresh
```

> **Real world:** Like a printed menu. If chef adds a new dish, you must **reprint** the menu. Old menus still show old items.

🌐 Networks + Volumes in Compose

yaml

```yaml
services:
  one:
    image: nginx
    ports:
```

```yaml
      - "8081:80"
    volumes:
      - volume-one:/usr/share/nginx/html  # data storage
    networks:
            - network-one                       # which lane it lives on


  two:
    image: httpd
    ports:
      - "8082:80"
    volumes:
      - volume-two:/usr/local/apache2/htdocs
    networks:
      - network-two

networks:               # declare your lanes
  network-one:
    driver: bridge
  network-two:
    driver: bridge

volumes:                # declare your storage boxes
  volume-one:
  volume-two:
```

Memory map:

```
volumes  = pen drive  (data saved even if container dies)
networks = phone lane (who can talk to who)
```

🗄️ Database Container with Volume

bash

```bash
# Step 1: Create a volume (storage box for DB data)

docker volume create database

# Step 2: Run MySQL container using that volume

docker run -d \
  --name database-cont \
  -v database:/var/lib/mysql \   # store DB files here
  -p 3306:3306 \                 # mysql port
```

```
  -e MYSQL_ROOT_PASSWORD=root \  # set password via env
  mysql/mysql-server:5.7

# Step 3: Login to container

docker exec -it database-cont /bin/bash

# Step 4: Login to MySQL

mysql -uroot -proot
```

Why `-e` for password?

```
-e = environment variable = accessible INSIDE container
ARG = only during BUILD time, not runtime
```

> Real world: `-e` is like telling the chef the secret recipe **while cooking**. `ARG` is only used **while setting up the kitchen**.

⏸️ Stop vs Down vs Pause — The 3 Brothers

```
┌──────────────────────────────────────────────┐
│  pause   = Freeze 🥶  (alive but not moving) │
│  stop    = Sleep 😴   (off, but not deleted) │
│  down    = Demolish 💣 (deleted everything)  │
└──────────────────────────────────────────────┘
```

Command	Container	Network	Volume
`pause`	Frozen	✅ exists	✅ exists
`stop`	Stopped	✅ exists	✅ exists
`down`	❌ deleted	❌ deleted	✅ exists

Bug fix workflow:

bash

```
docker compose down        # demolish

# fix your code

docker compose up -d --build  # rebuild fresh
```

📋 All Commands Cheatsheet

bash

```
docker compose up -d          # start all containers

docker compose up --build -d  # rebuild + start

docker compose down           # stop + delete containers

docker compose stop           # stop (keep containers)

docker compose start          # start stopped containers

docker compose pause          # freeze all

docker compose unpause        # unfreeze all

docker compose ps             # list running containers

docker compose logs           # see logs

docker compose images         # see images used

docker compose config         # show final parsed yaml

docker compose build          # only build, don't run
```

🧠 One-Line Memory Trick

> **"One YAML file. One command. Many containers. All at once."**

```
docker-compose.yaml  →  docker compose up -d  →  🎉 everything running
```

📁 Valid File Names (only these 4!)

```
docker-compose.yaml  ✅
docker-compose.yml   ✅
compose.yaml         ✅
compose.yml          ✅
```